

RAPPORT DE STAGE

Développement de la solution logicielle Scootup

Étudiant	Reda Mekaoui
Formation	BTS SIO — Option SLAM (2ème année)
Entreprise	OhLogiciel
Tuteur de stage	Oualid Handoura
Période	5 Janvier — 6 Février 2026
Lieu	Paris, Île-de-France

Sommaire

1. Résumé Exécutif

2. Introduction

- 2.1 Contexte de l'Entreprise
- 2.2 Problématique Urbaine
- 2.3 Objectifs du Stage

3. Présentation du Projet Scootup

- 3.1 Vision Produit
- 3.2 Architecture Globale du Système
- 3.3 Business Model

4. Missions et Responsabilités

- 4.1 Application Mobile Client
- 4.2 Interface d'Administration (Back-office)
- 4.3 Backend et Intégration IoT

5. Méthodologie de Travail

- 5.1 Organisation Agile
- 5.2 Gestion de Projet
- 5.3 Stratégie de Branches Git

6. Défis Techniques et Solutions

7. Tests et Qualité du Code

8. Chronologie Détaillée du Stage

9. Compétences Développées

10. Apports du Stage

11. Perspectives et Améliorations Futures

12. Conclusion

- A Annexe A — Technologies et Outils
- B Annexe B — Glossaire Technique
- C Annexe C — Résumé CV

1. Résumé Exécutif

Ce rapport présente mon expérience de stage de 5 semaines au sein de l'entreprise OhLogiciel, où j'ai développé Scootup, une solution innovante de mobilité urbaine destinée au stationnement sécurisé de trottinettes électriques à Paris.

En tant que développeur principal, j'ai conçu et réalisé l'intégralité de l'écosystème logiciel : application mobile client (React Native), interface d'administration back-office, et intégration backend avec les casiers connectés via une architecture IoT.

Livrable	Technologie	État
Application mobile client	React Native + Expo + TypeScript	Complété
Backend API REST	NestJS + Prisma + PostgreSQL	Complété
Filtrage géospatial	PostGIS (ST_DWithin, ST_Distance)	Complété
Intégration paiement	Stripe (SetupIntent, PaymentMethod)	Complété
Pipeline CI/CD	GitLab CI avec Docker	Complété
Tests E2E backend	Jest + Supertest (93.23% couverture)	Complété

2. Introduction

2.1 Contexte de l'Entreprise

OhLogiciel est une entreprise spécialisée dans le développement de solutions logicielles innovantes. Elle développe actuellement Scootup, un projet ambitieux visant à résoudre les problématiques de stationnement anarchique des trottinettes électriques dans les grandes métropoles françaises.

2.2 Problématique Urbaine

Le déploiement massif des trottinettes électriques en libre-service a créé de nouveaux défis urbains : encombrement des trottoirs, stationnement anarchique, dégradations, et problèmes d'accessibilité pour les personnes à mobilité réduite. Scootup propose une réponse structurée avec des casiers sécurisés et connectés.



Dimension	Espace minimal (Trottinette pliée)	Conseil
Longueur / Hauteur	115 cm	Pour la longueur totale de l'engin plié.
Largeur	25 cm	Si le guidon est plié ou étroit.
Largeur (Standard)	50 cm	Si le guidon ne se plie pas (cas le plus fréquent).
Épaisseur	50 cm	Pour l'encombrement du bloc colonne/guidon replié sur le plateau.

Prototype physique du casier Scootup et spécifications dimensionnelles

2.3 Objectifs du Stage

- Concevoir et développer une application mobile intuitive pour les utilisateurs finaux
- Créer une interface d'administration complète pour la gestion du parc de casiers
- Mettre en place l'architecture backend et l'intégration IoT avec les casiers connectés
- Appliquer les bonnes pratiques de développement (Clean Architecture, tests, CI/CD)
- Collaborer avec mon tuteur selon une méthodologie Agile

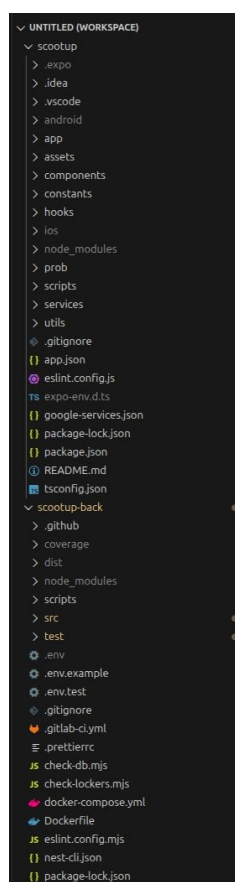
3. Présentation du Projet Scootup

3.1 Vision Produit

Scootup vise à déployer un réseau de stations équipées de casiers sécurisés et connectés permettant aux utilisateurs de localiser les stations disponibles, réserver un casier, gérer leur abonnement et déverrouiller leur casier via l'application mobile.

3.2 Architecture Globale du Système

Le projet repose sur trois composantes principales : l'application mobile client (React Native), le backend API (NestJS), et les casiers connectés avec verrouillage électronique.



Structure des 2 repositories :

- scootup — Frontend React Native (app, components, hooks, services)
- scootup-back — Backend NestJS (src, test, docker-compose, .gitlab-ci.yml)

Structure des deux repositories GitLab dans VS Code

3.3 Business Model

Le modèle économique repose sur des abonnements mensuels, une tarification à l'usage, des partenariats avec les opérateurs de trottinettes, et des subventions municipales.

BUSINESS MODEL

Qui ? : Scootup|

À qui ? : Les propriétaires de trottinettes électriques **ET** Les entreprises qui veulent proposer un parking sécurisé à leurs employés.

Quoi ? : Un casier physique blindé pour éviter le vol et le vandalisme.

Comment ? : Via une application mobile qui permet de géolocaliser, réserver, payer et ouvrir le casier via Bluetooth/NFC ou code.

Combien ? : l'abonnement/le prix de la location

Modèle économique :

Modèle économique de **plateforme** et d'**abonnement/location**

Business Model Canvas du projet Scootup

4. Missions et Responsabilités

4.1 Application Mobile Client

J'ai été responsable du développement complet de l'application mobile. Voici les spécifications fonctionnelles du cahier des charges :

I. Application Client

Le but de cette application est de fournir au client un moyen de mettre sa trottinette dans l'un de nos casier et de l'en sortir

Liste des fonctionnalités que l'application doit fournir:

1. Création de compte et login
2. Paiement de l'abonnement, suivi de consommation et désinscription
3. Carte avec la liste des stations et le nombre d'emplacements disponible par station
4. Possibilité de réserver une station par avance
5. Réclamation

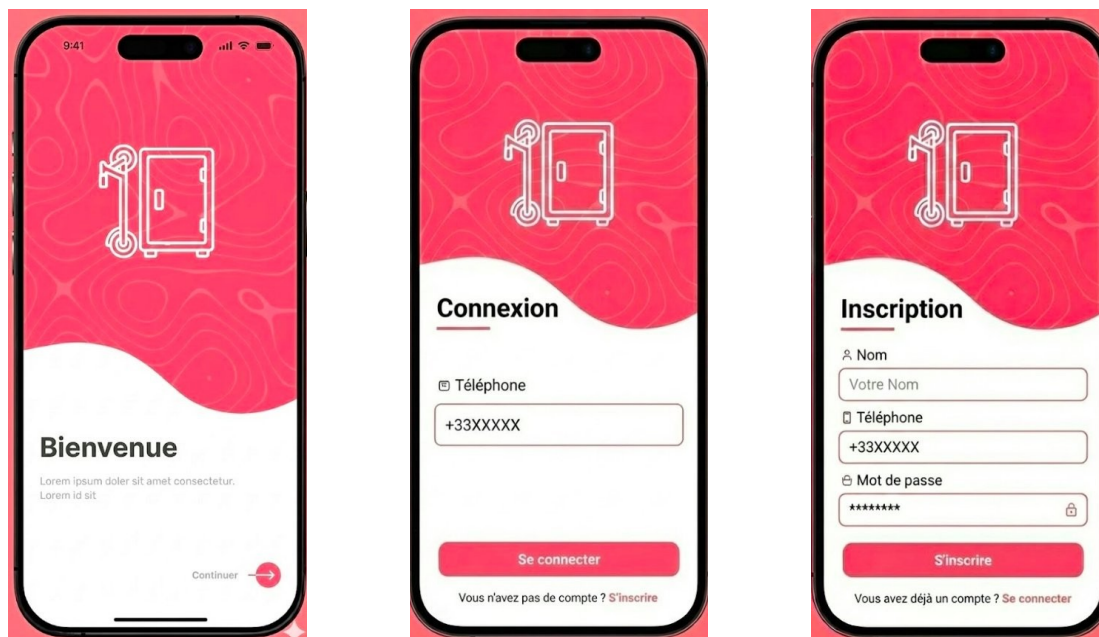
Spécifications fonctionnelles — Application Client

Stack Technique

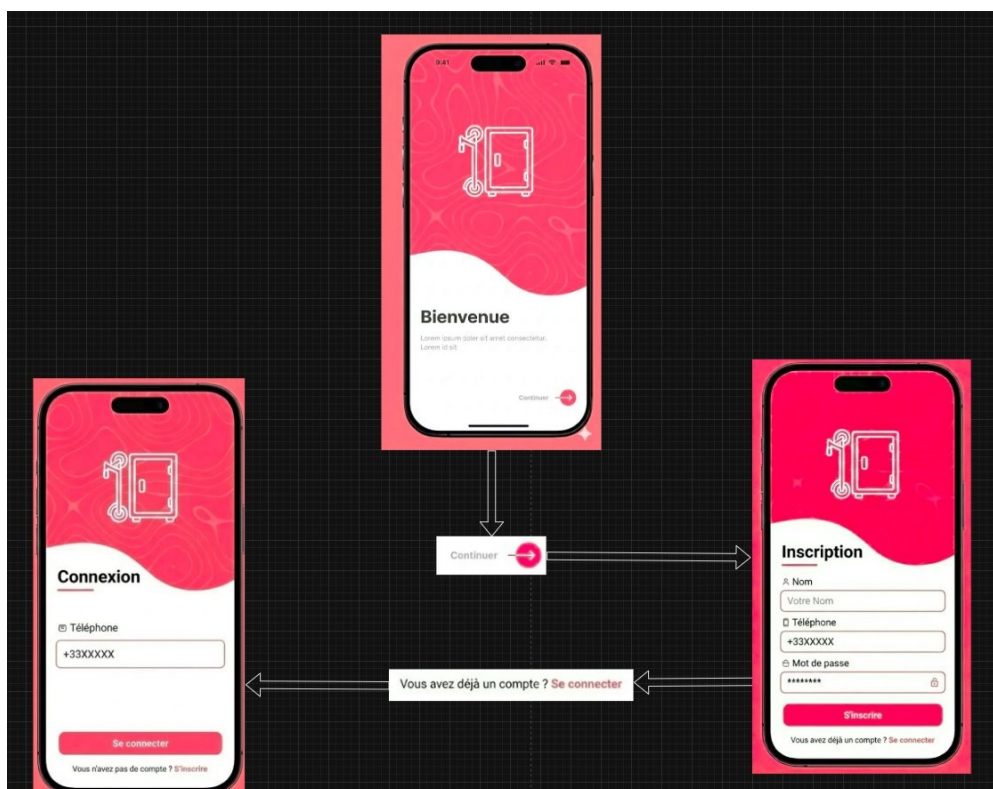
Technologie	Usage
React Native + Expo	Framework multiplateforme (Android / iOS)
TypeScript	Typage statique pour un code robuste
React Navigation	Navigation entre écrans (Bottom Tabs, Stack)
Google Maps API	Cartographie et géolocalisation des stations
Firebase Authentication	Authentification par code SMS
AsyncStorage	Persistance locale JWT et réservations
Axios	Client HTTP pour communication avec l'API

Maquettes Initiales (Version 1)

Dès le premier jour, j'ai réalisé les maquettes wireframes et l'arbre heuristique :

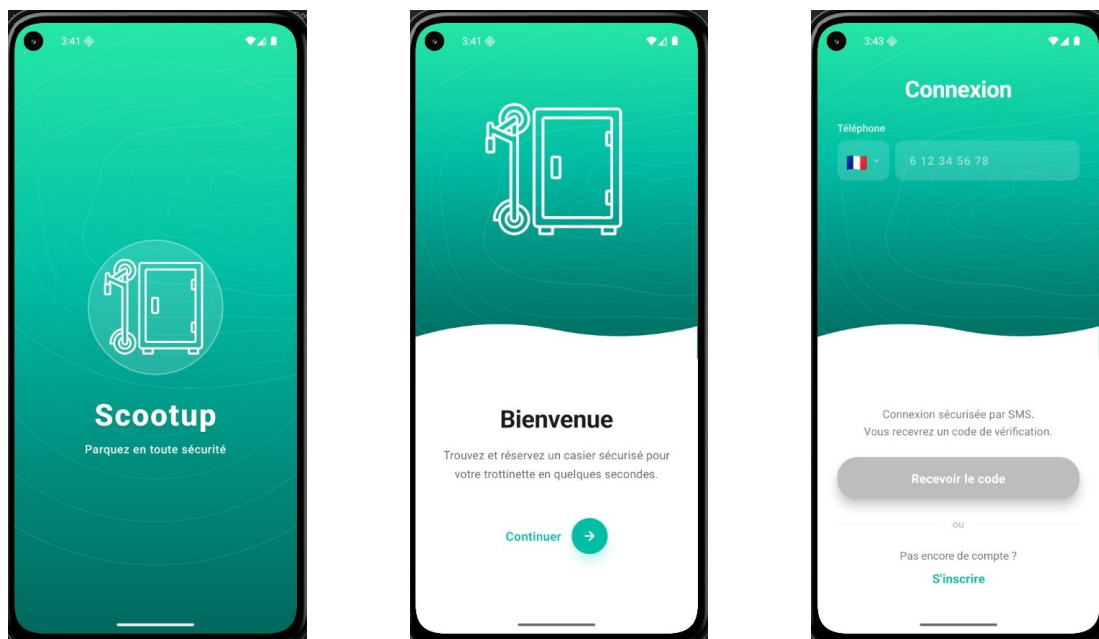


Maquettes v1 — Welcome, Connexion et Inscription (thème rose)

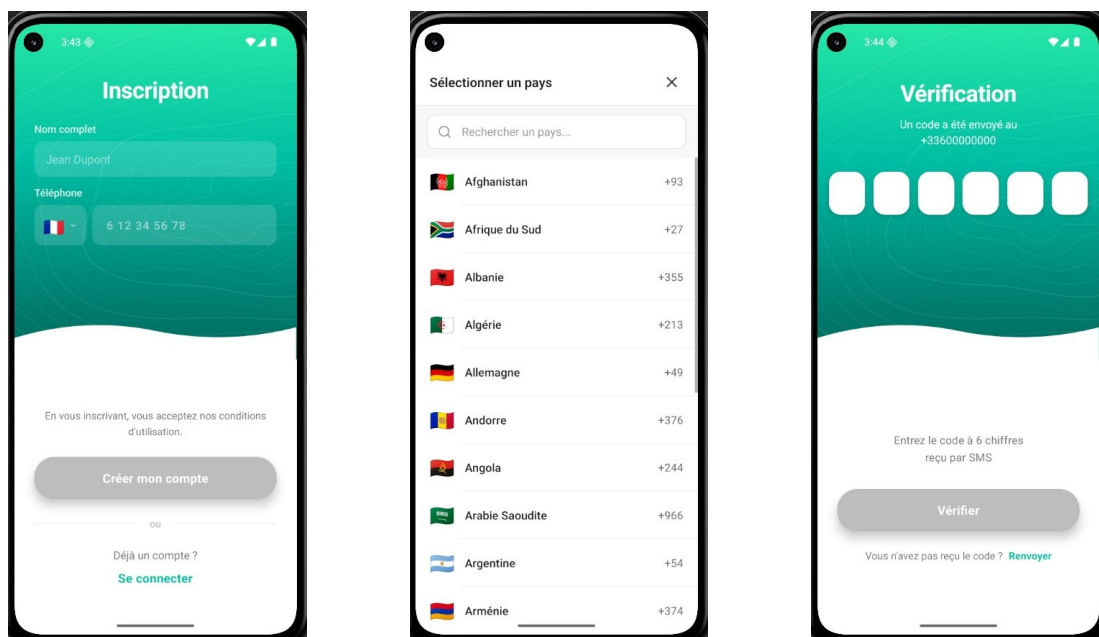


Arbre heuristique — Flux de navigation entre les écrans d'authentification

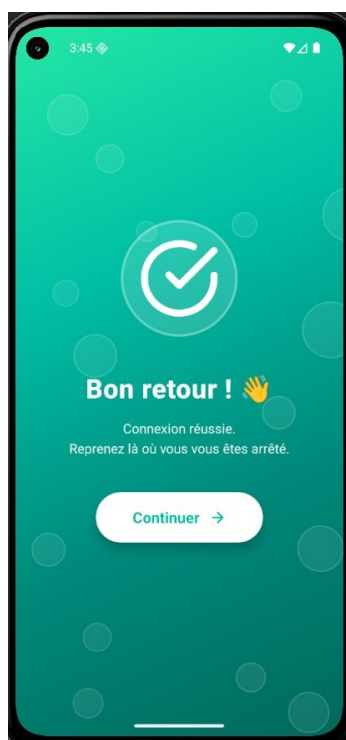
Application Finalisée — Authentification



Version finale — Splash Screen, Welcome et page de Connexion (thème vert)

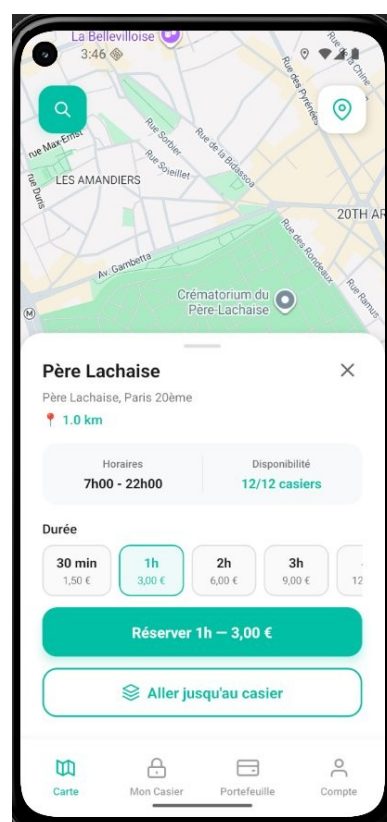
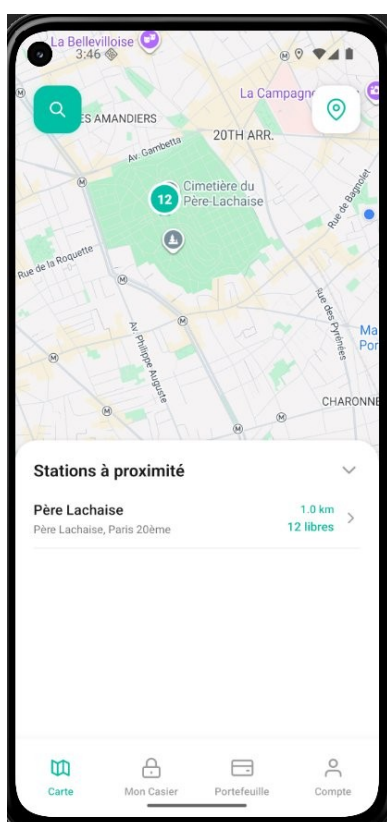


Inscription, Sélecteur de pays custom (185 pays) et Vérification SMS



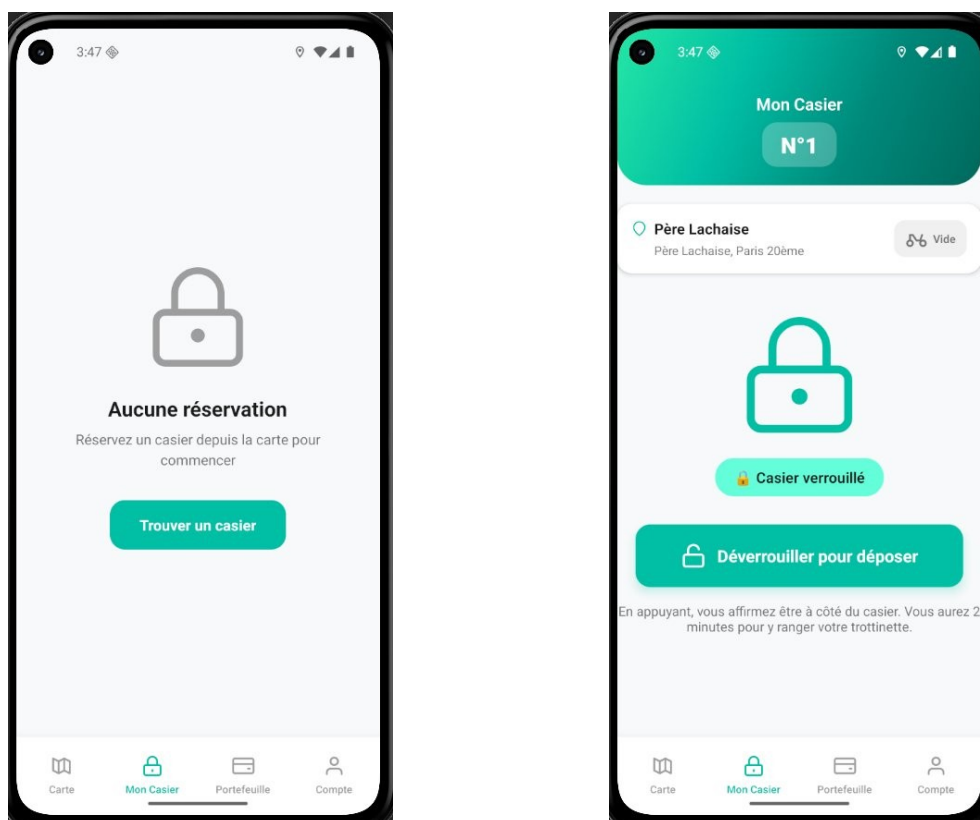
Écran de confirmation de connexion réussie

Application Finalisée — Carte Interactive



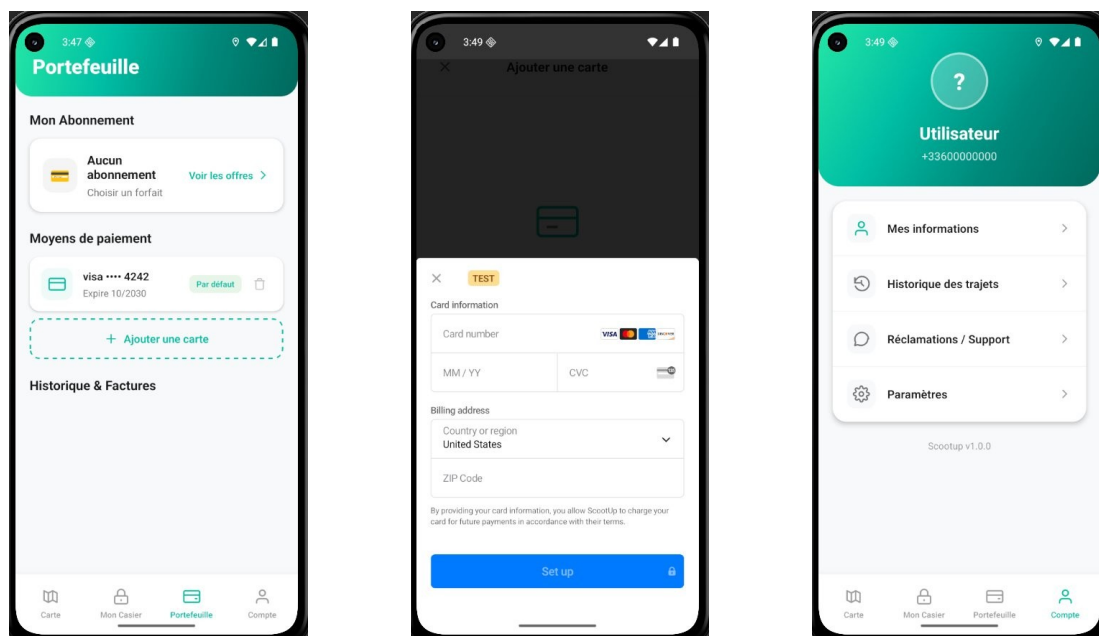
Carte avec stations à proximité et fiche station Père-Lachaise avec réservation

Application Finalisée — Gestion du Casier



Mon Casier — état vide (gauche) et casier N°1 actif avec bouton Déverrouiller (droite)

Application Finalisée — Portefeuille et Profil



Portefeuille avec carte Visa test, formulaire Stripe (mode test) et page Compte

4.2 Interface d'Administration (Back-office)

En parallèle, j'ai conçu l'interface d'administration permettant la supervision en temps réel, la gestion des stations et des utilisateurs, et les actions à distance sur les casiers.

II. Application Administrateur

Le but de cette application est d'avoir le statut de toutes les stations ainsi que leur historique et de pouvoir les gérer à distance.

Il doit être possible d'ouvrir un casier à distance, de mettre un casier unitaire ou toute la station inaccessible au client de fournir un token à la station pour que celle-ci puisse se synchroniser avec l'application ainsi que fournir toutes les routes pour ingérer la mise à jour des stations.

Liste des fonctionnalités que l'application doit fournir:

- a. Login via double authentification
- b. Suivi d'un client ou d'une station
- c. Etudier les réclamations des utilisateurs, pouvoir rembourser un utilisateur d'un montant libre
- d. Administration des clients
 - i. Remboursement etc
 - ii. Blocage
 - iii. Ouverture d'une station via un SMS
- e. Administration des stations
 - i. Bloquer une station ou un tiroir
 - ii. Mettre à jour une station

Spécifications fonctionnelles — Application Administrateur

4.3 Backend et Intégration IoT

J'ai mis en place une architecture backend robuste en Clean Architecture. Voici les spécifications IoT du cahier des charges :

III. Application embarquée

Le but de cette application est de faire l'interface client casier et de gérer les interactions avec le serveur.

Liste des fonctionnalités que l'application doit fournir

1. Dialoguer avec le casier pour Ouvrir/Fermer
2. Ecran client:
 - a. Ouvrir un casier pour verrouiller/déverrouiller une trotinette
 - i. Via NFC
 - ii. Via login et code SMS
3. Envoyer le statut de la station toutes les minutes
 - a. Nombre de casier libre
 - b. Nombre de casier occupé
 - c. Nombre de casier indisponible
4. Écouter un appel pour déverrouiller un casier à distance (voir la sécurité)
5. Envoyer une alerte en cas de problème

Spécifications fonctionnelles — Application Embarquée (IoT)

Stack Technique Backend

Technologie	Usage
NestJS 10.x	Framework TypeScript pour serveur scalable
Prisma ORM 5.x	Accès type-safe à la base de données
PostgreSQL 15+ + PostGIS	BDD relationnelle avec extension géospatiale
Firebase Admin SDK	Vérification des tokens SMS
Stripe API	Paievements (SetupIntent, PaymentMethod)
JWT / jsonwebtoken	Authentification stateless sécurisée
Docker / Docker Compose	Conteneurisation de PostgreSQL + PostGIS

Schéma de Base de Données

Table	Description
user	Informations utilisateurs (googleId, phone, name, timestamps)
locker	Stations de casiers (localisation géographique, capacité, statut)
slot	Casiers individuels (numéro, disponibilité, statut maintenance)
reservation	Réservations actives et historique
actionlog	Journal d'audit (reserve, unlock, lock, cancel, complete)
paymentmethod	Moyens de paiement enregistrés (Stripe)

5. Méthodologie de Travail

5.1 Organisation Agile

- Sprints d'une semaine avec objectifs définis chaque lundi
- Réunions quotidiennes rapides (stand-up) avec mon tuteur
- Revues de code régulières avec retours détaillés
- Itérations basées sur les retours et les tests

5.2 Gestion de Projet

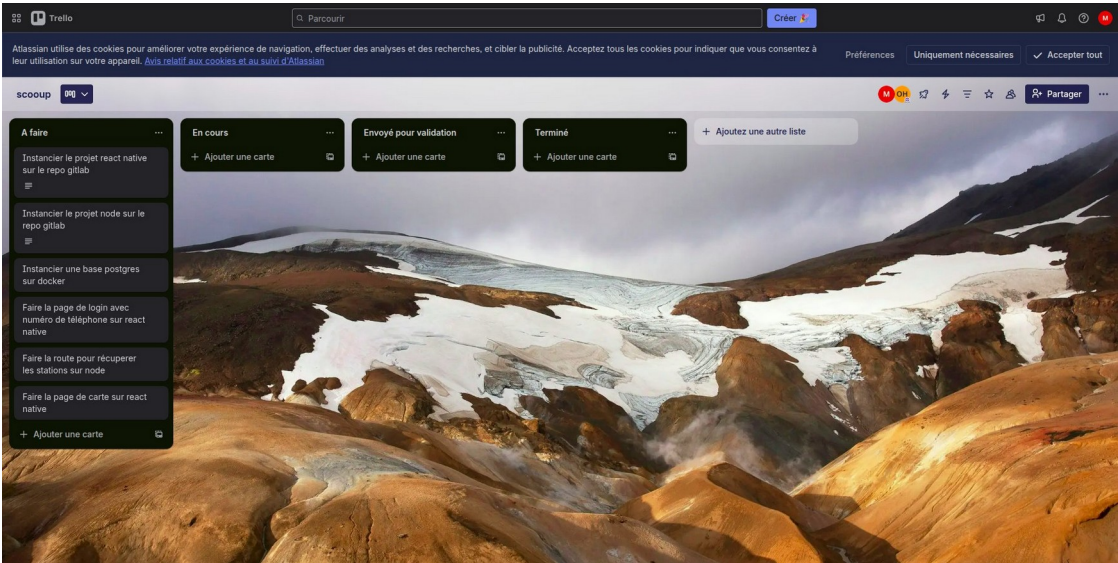
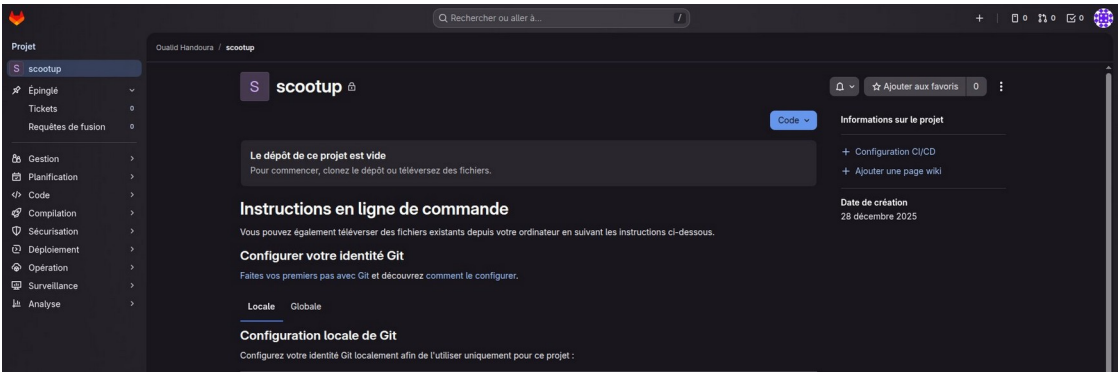


Tableau Trello — Kanban du projet Scoopup avec les premières tâches



Dépôt GitLab — Repository scoopup à l'initialisation

5.3 Stratégie de Branches Git

Branche	Usage
main	Production stable — merge conditionnel uniquement
develop	Intégration pour le développement continu
epic/payment	Branches epic pour les fonctionnalités majeures

feat/feature-name	Branches feature pour les développements spécifiques
-------------------	--

6. Défis Techniques et Solutions

Défi 1 : Persistance de la Session Utilisateur

Problème : Le token JWT était uniquement en mémoire — l'utilisateur devait se reconnecter à chaque rechargement.

Solution : Sauvegarde du JWT dans AsyncStorage, chargement automatique au démarrage et vérification de validité avant affichage du splash screen.

Résultat : L'utilisateur atterrit directement sur la carte sans reconnexion.

Défi 2 : Visibilité des Pins sur Google Maps

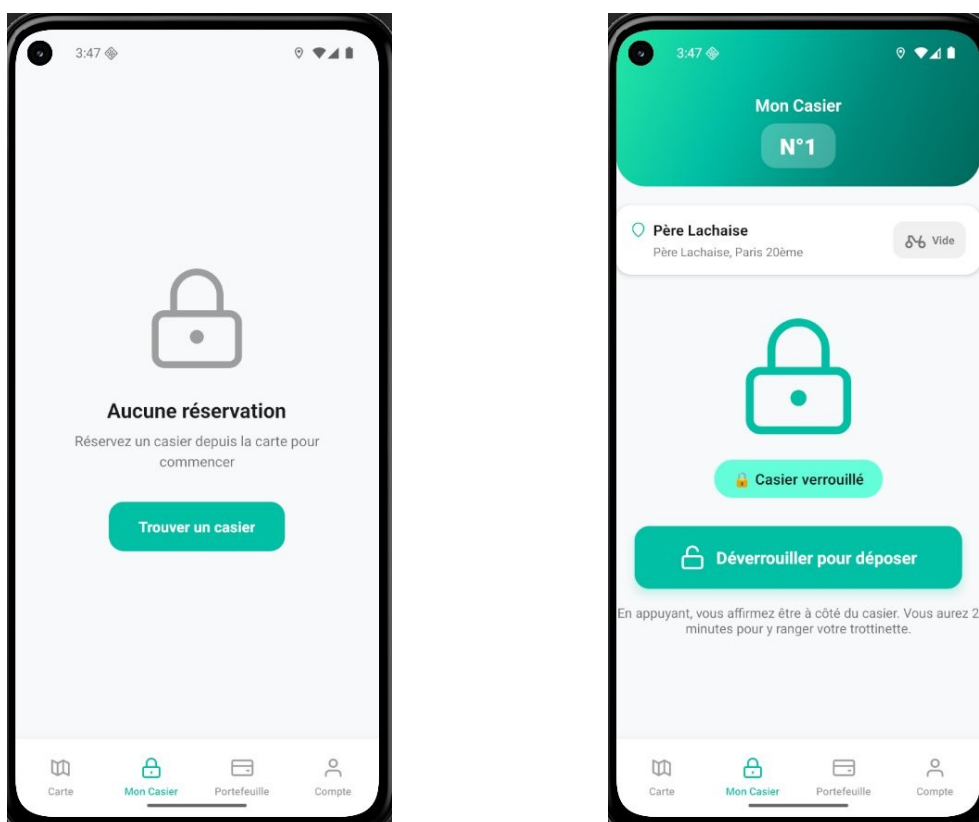
Problème : Les marqueurs étaient décalés par des propriétés de positionnement incorrectes (anchor, centerOffset).

Solution : Suppression de ces propriétés, pin agrandi à 56×56px, ajout d'ombre portée. Les 12 casiers de la station Père-Lachaise sont maintenant visibles sur la carte.

Défi 3 : Gestion du Flux de Réservation

Problème : L'interface ne respectait pas le flux — déverrouillage pour dépôt (2 min) → attente → récupération avec confirmation.

Solution : Machine à états (empty / deposited), compte à rebours de 2 minutes, badge "Trottinette déposée" et modal de confirmation avec temporisation de 5 secondes anti-erreur.



État PENDING (casier vide) et état ACTIVE (bouton Déverrouiller pour déposer)

Défi 4 : Filtrage Géospatial avec PostGIS

Problème : Filtrer les stations par proximité de manière performante, sans calculs JavaScript côté serveur.

Solution : Extension PostGIS, types géométriques POINT dans Prisma, requêtes SQL brutes avec ST_DWithin et index GIST spatial.

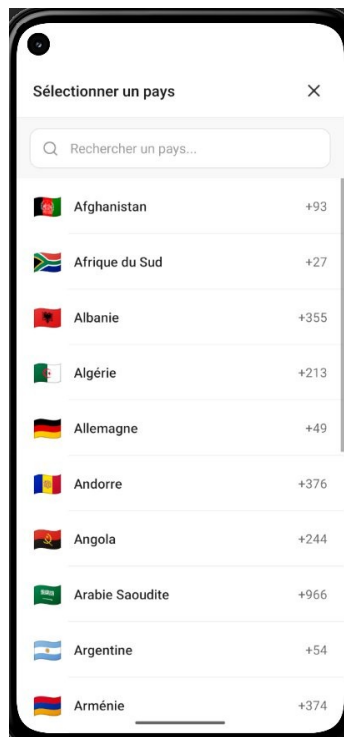
```
-- Requête PostGIS pour stations à proximité
SELECT id, name, latitude, longitude,
       ST_Distance(location::geography,
                   ST_SetSRID(ST_MakePoint($2, $1), 4326)::geography) AS distance
FROM locker
WHERE ST_DWithin(location::geography,
                 ST_SetSRID(ST_MakePoint($2, $1), 4326)::geography, $3)
ORDER BY distance ASC LIMIT 20;
```

Résultat : 15ms de temps de réponse (vs +200ms en JavaScript pur).

Défi 5 : Sélecteur de Pays Personnalisé (PhoneInput)

Problème : La bibliothèque react-native-country-picker-modal avait un modal Android invisible, une recherche cassée et une taille de 150KB.

Solution : Composant PhoneInput.tsx custom — 185 pays, modal natif, recherche temps réel, formatage automatique, 15KB seulement.



Sélecteur de pays custom — 185 pays avec drapeaux emoji et recherche en temps réel

Défi 6 : Pipeline CI/CD GitLab

Problème : Automatiser les tests et déploiements pour garantir la qualité à chaque merge request.

Solution : Pipeline .gitlab-ci.yml (build → test → deploy). Après débogage de 4 erreurs et optimisations (cache, parallélisation), le pipeline passe de 4m10s à 2m23s.

```
Stage build : ✓ 48s (après optimisation du cache)
Stage test  : ✓ 1m35s (après parallélisation --maxWorkers=4)
Tests: 78 passed, 78 total | Coverage: 93.23%
```

7. Tests et Qualité du Code

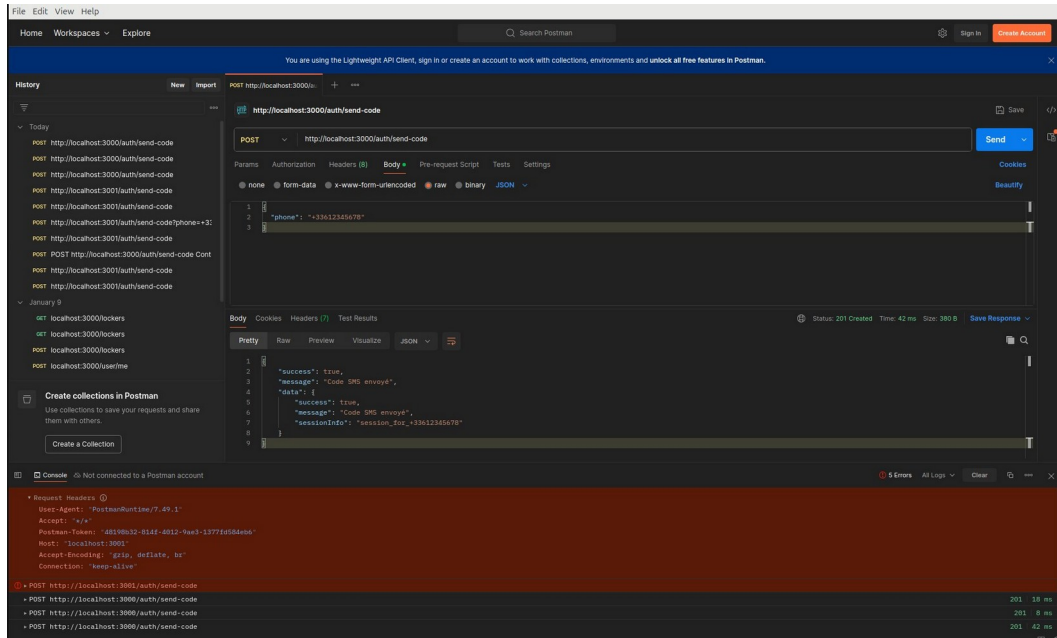
7.1 Stratégie de Test Backend

- Tests d'authentification : vérification des tokens, connexion, inscription
- Tests des réservations : création, déverrouillage, annulation, complétion
- Tests des casiers : liste, filtrage par distance GPS, disponibilité
- Tests de santé : health check et connexion base de données

Module	Statements	Branches	Functions	Lines
Tous les fichiers	93.23%	64.39%	92.66%	92.83%
domain/bookings/use-cases	93.15%	56.25%	100%	92.30%
domain/payment/use-cases	100%	50%	100%	100%
domain/users/use-cases	96.29%	57.14%	100%	95.65%
infrastructure/repositories	87.50%	76.92%	84.61%	88.00%
presentation/rest	95.83%	83.33%	92.30%	96.15%

Évolution : 38 tests à 88.71% → 78 tests à 93.23% (+40 tests, +4.52% de couverture).

7.2 Tests via Postman



Postman — Test de l'endpoint POST /auth/send-code (réponse 201 Created)

7.3 Bonnes Pratiques Appliquées

- Clean Architecture : séparation Domain / Infrastructure / Presentation
- Type Safety : zéro any dans le code de production (TypeScript strict)
- Validation des DTOs : class-validator pour toutes les entrées HTTP

- Logging structuré : traçabilité complète dans la table actionlog
- ESLint : passage de 91 erreurs à 0 en fin de stage

8. Chronologie Détaillée du Stage

Journal de bord condensé par semaine avec les décisions techniques importantes et les blocages résolus.

Semaine 1 (5–9 Janvier) : Prise en Main et Initialisation

Lundi 5 — Configuration SSH et clonage du dépôt GitLab :

```
reda@reda-IdeaPad:~/Projects$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 [redacted]
reda.[redacted]@gmail.com
reda@reda-IdeaPad:~/Projects$ git clone git@gitlab.com:ohandoura/scootup.git
Clonage dans 'scootup'...
warning: Vous semblez avoir cloné un dépôt vide.
reda@reda-IdeaPad:~/Projects$ cd scootup
reda@reda-IdeaPad:~/Projects/scootup$
```

Terminal — Configuration SSH et clonage du dépôt scootup

Analyse du cahier des charges et élaboration de l'arborescence de l'application :

1. Zone Publique (Non connecté)

- **Splash Screen** (Logo Scootup au chargement)
- **Page d'Accueil / Onboarding** (Présentation rapide du concept ou phrase d'accroche)
- **Login** (Email + Mot de passe)
 - Lien : Mot de passe oublié
- **Register** (Création de compte)
 - Étape validation : Saisie du code SMS

- 2. Zone Connectée**
- Une fois connecté, l'utilisateur arrive sur la **Vue Principale (Map)**. L'appi s'articule autour de 4 onglets principaux en bas de l'écran.
- Onglet A : Carte / Stations (Page d'accueil par défaut)**
- **Vue Carte** (Google Maps avec les points)
 - Barre de recherche (Trouver une adresse)
 - **Fiche Station** (S'ouvre quand on clique sur un point)
 - Infos (Adresse, Horaires)
 - Disponibilité (Combien de casiers libres ?)
 - Bouton : **Réserver** → Ouvre le tunnel de réservation
 - Écran : Confirmation de réservation (Choix durée si besoin)
 - Écran : Succès ("Votre casier est réservé pour 15min le temps d'arrêter")
- Onglet B : Mon Casier (L'action immédiate)**
- Si aucune réservation : Message "Aucune location en cours" + Bouton "Trouver une station".
 - Si réservation active :
 - Numéro du casier attribué.
 - Chronomètre (Temps restant ou durée en cours).
 - Gros Bouton : **DÉVERGOLIER / OUVRIER** (Action IoT).
 - Bouton : Terminer la location (Libérer le casier).
- Onglet C : Portefeuille (Abonnement)**
- **Tableau de bord Conso** : Graphique ou résumé (Heures consommées ce mois-ci).
 - **Mon Abonnement** :
 - Détails de l'offre actuelle.
 - Bouton "Changer d'offre / Se désabonner".
 - **Moyens de paiement** :
 - Liste des cartes enregistrées.
 - Ajouter une carte.
 - **Historique / Factures** : Liste des paiements passés.
- Onglet D : Compte & Support**
- **Mes Informations** : (Nom, Email, Tel).
 - **Historique des trajets** : Liste des anciennes locations (Date, Station, Durée).
 - **Réclamations / Support** :
 - Liste des tickets ouverts (Statut).
 - Bouton : **Nouvelle réclamation** (Formulaire : Choix du motif → Message → Envoyer).
 - **Paramètres** : (Notifications, Langue).
 - **Déconnexion**.

Arborescence Zone Publique (non connecté) et Zone Connectée

Mardi 6 — Blocage sur les droits GitLab. Décision architecturale : séparation en 2 repositories (Frontend / Backend). Travail en données mockées en attendant.

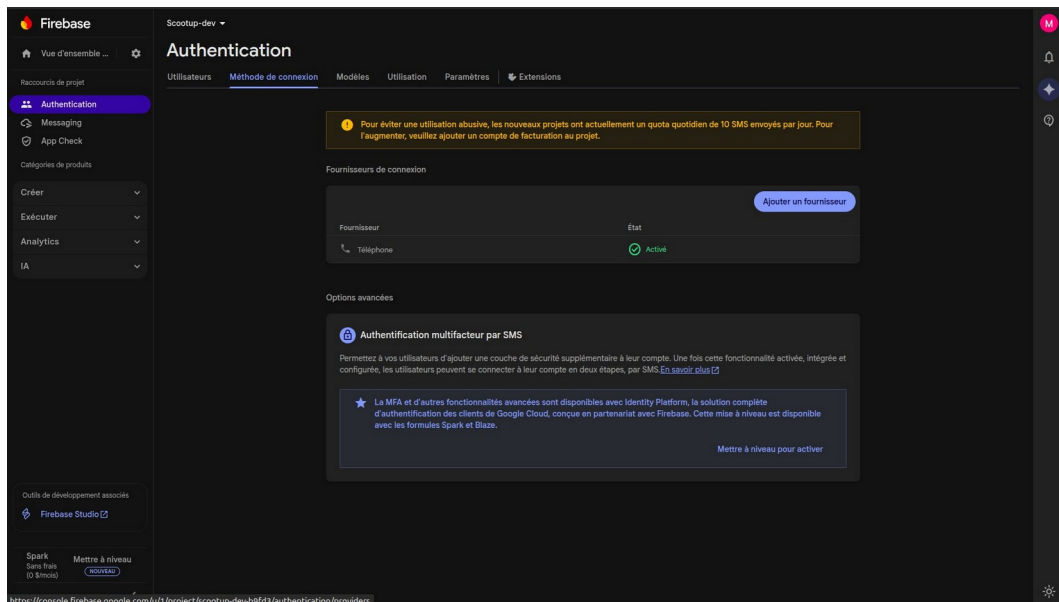
Mercredi 7 — Développement du Business Model et des 3 pages d'authentification (première version rose/blanc).

Jeudi 8 — Changement de palette vers le thème vert actuel, création du Splash Screen et des pages Carte, Mon Casier, Portefeuille, Profil.

Vendredi 9 — Initialisation du backend NestJS, PostgreSQL via Docker Compose, Prisma ORM.

Semaine 2 (12–16 Janvier) : Développement des Fonctionnalités Core

Lundi 12 — Configuration Firebase pour l'authentification par SMS :



Console Firebase — Activation de l'authentification par téléphone (projet scootup-dev)

Flux complet implémenté : client → /auth/send-code → Firebase → SMS → /auth/verify-code → JWT. Blocages résolus : credentials Firebase mal configurés, types TypeScript incohérents, JWT Secret manquant.

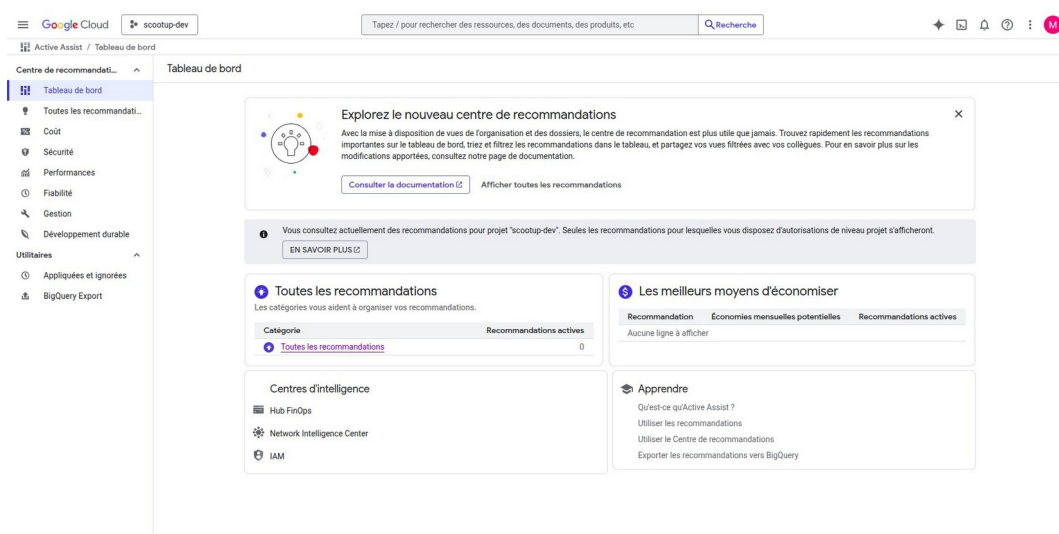
Mardi 13 — Règle "zéro any", DTOs avec class-validator. Premiers tests E2E.

Mercredi 14 — Service API centralisé frontend (services/api.ts). Endpoint GET /lockers côté backend.

Jeudi 15 — Intégration frontend-backend complète, remplacement de tous les mocks par de vrais appels API.

Semaine 3 (19–23 Janvier) : Intégration et Système de Réservation

Lundi 19 — Création du compte Google Cloud Platform, activation des Maps SDK :



Console Google Cloud Platform — Projet scootup-dev

Mardi 20 — Google Maps complet. Formule de Haversine et CheckReservationDistanceUseCase (500m max). Blocages : carte blanche Android, permission iOS refusée en simulateur.

Mercredi 21 — Composant PhoneInput custom (voir Défi 5).

Jeudi 22 — Système de réservation complet : tables reservation et actionlog, 8 use cases, transactions Prisma, 15 tests unitaires.

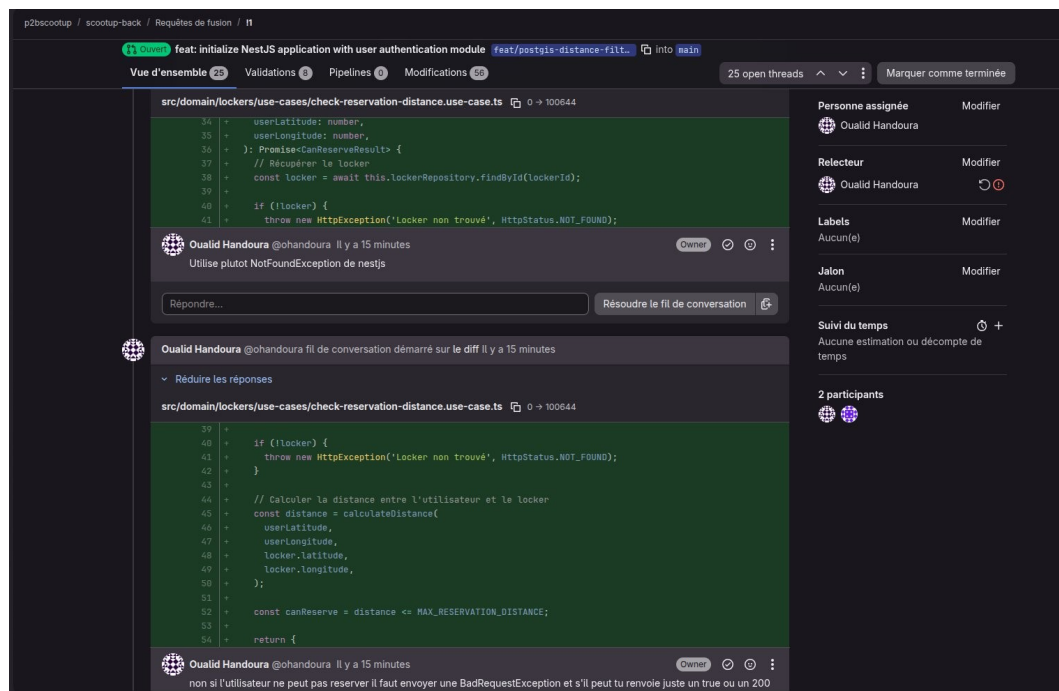
Vendredi 23 — Refonte page "Mon Casier" avec machine à états.

Semaine 4 (26–30 Janvier) : CI/CD et Améliorations

Lundi–Mercredi — Pipeline GitLab CI/CD : 4 erreurs déboguées, réduction de 4m10s à 2m23s.

Jeudi 29 — Tests unitaires frontend (PhoneInput, useReservation, ApiService). Pipeline mobile configuré.

Vendredi 30 — Code review complète avec le tuteur (47 fichiers). Identification de 12 améliorations mineures, documentation technique rédigée :



Merge Request GitLab — Code review du tuteur Oualid Handoura avec commentaires sur les commits

Semaine 5 (2–6 Février) : Module Paiement et Finalisation

Mardi 3 — Code review PR feat/reservation-system. Optimisation slot retrieval de 87ms à 23ms. PostGIS avec index GIST.

Mercredi 4 — Fusion PostGIS sur main. Module Stripe : table paymentmethod, StripeService, 3 use cases, 3 endpoints.

Jeudi 5 — Intégration Stripe frontend. +40 tests → 78 tests à 93.23%. 91 erreurs ESLint → 0.

Vendredi 6 — Finalisation, documentation, nettoyage Git, Trello. Réunion de bilan avec le tuteur.

9. Compétences Développées

9.1 Compétences Techniques

Frontend

- Maîtrise de React Native et Expo pour le développement mobile multiplateforme
- TypeScript avancé pour un code robuste et maintenable
- Intégration APIs externes (Google Maps, Firebase, Stripe)
- Hooks personnalisés complexes (useReservation, useAuth)

Backend

- Clean Architecture et SOLID appliqués en NestJS
- Prisma ORM avec PostGIS pour requêtes géospatiales
- Authentification JWT et intégration Stripe sécurisée

DevOps

- Pipeline CI/CD GitLab avec Docker et parallélisation
- Tests E2E avec couverture de code (Jest, Supertest)
- Gestion Git avec stratégie de branches rigoureuse

9.2 Compétences Transversales

- Autonomie : gestion indépendante du développement full-stack complet
- Résolution de problèmes : 6 défis techniques majeurs résolus
- Méthodologie Agile : sprints hebdomadaires, stand-ups, code reviews
- Communication : échanges réguliers avec le tuteur pour validation

10. Apports du Stage

10.1 Apports Professionnels

- Compréhension des enjeux business et techniques d'un projet IoT
- Développement d'une application mobile de A à Z
- Architecture backend scalable en situation réelle
- Pratique Agile avec un tuteur expérimenté

10.2 Apports Pédagogiques

- Application concrète de la Clean Architecture
- Mise en pratique des bases de données relationnelles (SQL, Prisma)
- Utilisation professionnelle de Git en environnement collaboratif
- Découverte de technologies non enseignées (PostGIS, Stripe, Firebase)

10.3 Apports Personnels

- Gain de confiance dans mes capacités de développement
- Développement de l'autonomie technique et de la rigueur
- Confirmation de mon projet professionnel vers l'ingénierie logicielle

11. Perspectives et Améliorations Futures

11.1 Fonctionnalités à Implémenter

- Finalisation du module de paiement Stripe (abonnements, historique)
- Notifications push pour alertes en temps réel
- Programme de fidélité et réductions pour utilisateurs réguliers
- Mode hors-ligne avec synchronisation différée

11.2 Améliorations Techniques

- Migration vers React Query pour gestion optimisée du cache
- WebSockets pour mises à jour temps réel des disponibilités
- Documentation API avec Swagger/OpenAPI
- Monitoring avec Sentry ou DataDog

11.3 Déploiement Production

- Déploiement backend sur Google Cloud Run
- Publication sur App Store et Google Play
- Stratégie de rollback et feature flags

12. Conclusion

Ce stage de 5 semaines au sein d'OhLogiciel a été une expérience particulièrement enrichissante et formatrice. J'ai eu l'opportunité de travailler sur un projet complet et ambitieux, de la conception à l'implémentation, en passant par les tests et le déploiement.

Le développement de Scootup m'a permis de mettre en pratique l'ensemble de mes compétences full-stack, tout en découvrant de nouvelles technologies comme PostGIS, NestJS et Stripe. La Clean Architecture a guidé chaque décision de conception.

Au-delà des aspects techniques, ce stage m'a appris l'importance de la rigueur : gestion de version, tests automatisés, code reviews et documentation. Le passage de 91 erreurs ESLint à 0, de 38 tests à 78, et d'une couverture de 88% à 93% en témoigne.

Cette expérience confirme mon souhait de poursuivre en école d'ingénieur pour approfondir mes connaissances en architecture logicielle, intelligence artificielle et systèmes distribués.

Je tiens à remercier sincèrement mon tuteur Oualid Handoura pour son accompagnement, sa confiance et ses précieux conseils tout au long de ce stage.

Annexes

Annexe A : Technologies et Outils Utilisés

Catégorie	Technologies
Frontend	React Native 0.72+, Expo SDK 49+, TypeScript 5.0+, React Navigation 6.x, Google Maps API, Firebase Auth, AsyncStorage, Axios, Expo Location
Backend	NestJS 10.x, Prisma ORM 5.x, PostgreSQL 15+, PostGIS, Firebase Admin SDK, Stripe API, JWT, class-validator, bcrypt, Node.js 18+
DevOps	Git / GitLab, Docker / Docker Compose, GitLab CI/CD, Trello, VS Code, Postman, Expo Go, Jest, ESLint, Prettier
Cloud	Google Cloud Platform, Firebase, Stripe

Annexe B : Glossaire Technique

Terme	Définition
API REST	Architecture permettant la communication entre applications via HTTP
Clean Architecture	Séparation des couches métier / infrastructure / présentation
JWT	JSON Web Token — Standard d'authentification sécurisé sans état
PostGIS	Extension PostgreSQL pour données géospatiales (distances, proximité)
CI/CD	Continuous Integration/Deployment — Automatisation tests et déploiements
IoT	Internet of Things — Objets connectés communiquant via Internet
SetupIntent	Objet Stripe pour enregistrer une carte sans paiement immédiat
GIST	Index spatial PostgreSQL permettant des requêtes géographiques ultra-rapides

Annexe C : Résumé CV

*Stagiaire Développeur Fullstack | OhLogiciel (5 semaines — Jan–Fév 2026)
Développement complet de Scootup : application mobile React Native, API NestJS/Prisma/PostGIS, intégration IoT et paiement Stripe. Clean Architecture, CI/CD GitLab, 78 tests E2E à 93.23% de couverture, déploiement Google Cloud Platform.*